Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Complete checklist and key steps for performing a **SQL Server on Azure VM Health Check** — focused on **optimal performance, stability, and cost efficiency** 👇

🔍 1. VM-Level Health Check

✅ 1.1. VM Size & SKU

- Verify the **VM size matches workload** (vCore count, memory-to-core ratio, and storage throughput).
- Example: Use **E-series** (memory-optimized) for OLTP and **D-series** for general workloads.
- Compare actual CPU utilization vs provisioned cores.

Query:

```
SELECT cpu_count, physical_memory_kb/1024/1024 AS MemoryGB
FROM sys.dm_os_sys_info;
```

✅ 1.2. OS & Patch Compliance

- Ensure latest **Windows updates** and **SQL Server Cumulative Updates (CU)** are applied.
- Check for **firmware, drivers, and network agents** updates in Azure.

✅ 1.3. Azure VM Storage Configuration

- Use **Premium SSD or Ultra Disk** for data, log, and tempdb.
- Avoid placing data/log/tempdb on the same drive.
- Validate disk caching:
  - **ReadOnly** for data.
  - **None** for logs and TempDB.

PowerShell:

```
Get-AzVM -ResourceGroupName <RGName> -Name <VMName> | Get-AzVMDiskEncryptionStatus
```


⚙️ 2. SQL Server Instance-Level Health Check

✅ 2.1. Max Memory & Parallelism

- **Max Server Memory**: Leave ~2–4 GB for OS.
- **MAXDOP**: Follow Microsoft recommendation:
  - <=8 cores → MAXDOP = number of cores
  - 8 cores → MAXDOP = 8
- **Cost Threshold for Parallelism**: Start from 30.

Query:

```
EXEC sp_configure 'show advanced options', 1; RECONFIGURE;
EXEC sp_configure;
```

✅ 2.2. TempDB Configuration

- Multiple data files = number of logical processors / 4 (up to 8 files to start).
- All files same size & autogrowth.
- Place on **dedicated Premium SSD**.

Query:

```
SELECT name, size/128 AS SizeMB, growth, physical_name
FROM sys.master_files WHERE database_id = 2;
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

✅ 2.3. Instant File Initialization (IFI)

- Ensure **"Perform Volume Maintenance Tasks"** right is granted to SQL Service account.
- Speeds up file growth and database creation.

✅ 2.4. Database Settings

- Auto-growth increment: Fixed MBs (not %).
- Recovery model appropriate (Full vs Simple).
- Page verification = CHECKSUM.
- Compatibility level = latest supported.

📊 3. Performance Monitoring & Bottleneck Check

✅ 3.1. CPU Usage

- Identify top CPU consumers:

```
SELECT TOP 10 total_worker_time/1000 AS CPU_ms, execution_count,
    total_worker_time/execution_count AS AvgCPU,
    DB_NAME(database_id) AS DBName,
    OBJECT_NAME(object_id, database_id) AS ProcName
FROM sys.dm_exec_query_stats
CROSS APPLY sys.dm_exec_sql_text(sql_handle)
ORDER BY total_worker_time DESC;
```

✅ 3.2. Memory Pressure

- Check Buffer Pool usage and Page Life Expectancy:

```
SELECT cntr_value AS PageLifeExpectancy
FROM sys.dm_os_performance_counters
WHERE counter_name = 'Page life expectancy';
```

- If PLE < 300 → Memory pressure likely.

✅ 3.3. IO & Storage Throughput

- Review I/O latency using sys.dm_io_virtual_file_stats:

```
SELECT DB_NAME(database_id) AS DBName, file_id,
    io_stall_read_ms/num_of_reads AS AvgReadLatency_ms,
    io_stall_write_ms/num_of_writes AS AvgWriteLatency_ms
FROM sys.dm_io_virtual_file_stats(NULL, NULL)
ORDER BY AvgReadLatency_ms DESC;
```

- Healthy disks: < 5–10 ms latency.

✅ 3.4. Wait Statistics

- Identify top wait types to diagnose performance blockers:

```
SELECT TOP 10 wait_type, wait_time_ms/1000.0 AS WaitTime_s,
    signal_wait_time_ms/1000.0 AS SignalWait_s
FROM sys.dm_os_wait_stats
ORDER BY wait_time_ms DESC;
```

- Focus on **PAGEIOLATCH**, **CXPACKET**, **LCK_M**, etc.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

🧠 4. Network & Connectivity

✅ 4.1. Azure Network Latency

- Use **Azure Network Watcher** or **Latency test** between app and DB VM.
- Ensure SQL port (1433) not filtered by NSG/firewall.

✅ 4.2. NIC Throughput & Drivers

- Validate NIC settings in VM: Accelerated Networking enabled.
- Ensure no packet drops or retransmissions.

💾 5. Backup & Maintenance Strategy

✅ 5.1. Backup Validation

- Verify automated backups (Azure Backup or custom jobs) are working.
- Test restore to another Azure VM or SQL instance.

✅ 5.2. Index & Statistics Maintenance

- Regular rebuild/reorganize.
- Update stats with FULLSCAN for critical OLTP systems.

🧩 6. Azure Integration & Cost Optimization

✅ 6.1. Azure Hybrid Benefit

- Ensure you're using **AHB** if you have SQL Server licenses with SA.

✅ 6.2. Azure Advisor Recommendations

- Review Advisor for:
  - Underutilized VM (resize or shutdown)
  - Reserved Instance opportunities
  - Disk throttling alerts

📈 7. Reporting & Automation

- Schedule weekly or monthly health checks using:
  - PowerShell scripts
  - SQL Agent Jobs
  - Azure Automation Runbooks
- Generate **HTML/PDF reports** summarizing:
  - CPU, Memory, I/O, Wait stats, Missing indexes, Blocking, TempDB usage.

⚡ Proactive Tuning Tips

- Enable **Query Store** to track regressions.
- Review top **resource-consuming queries** weekly.
- Implement **Automatic tuning** (where applicable).
- Use **Azure Monitor / Log Analytics** for continuous insights.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Rready-to-use **SQL Server on Azure VM Health Check script pack** (PowerShell + T-SQL) you can run right away. It will:

- Run key instance, database, TempDB, IO, wait-stats, top-queries, index & stats, and backup checks.
- Export results to CSV and a single HTML summary report (easy to share).
- Be runnable from an admin workstation or the Azure VM itself (uses Invoke-Sqlcmd from the SqlServer PowerShell module).

Below are the scripts and instructions. Copy the files into a folder (eg. C:\SqlHealthCheck) and run the PowerShell Run-HealthCheck.ps1 with appropriate credentials.

How to use (quick)

1. Create a folder, paste the files below as described.
2. On the machine where you'll run it (Azure VM or admin host):
    - Install the SqlServer PowerShell module if not present:
    - Install-Module SqlServer -Scope CurrentUser -Force
    - Ensure account used has sysadmin on the SQL instance (or sufficient rights to query DMVs).
3. Edit variables at top of Run-HealthCheck.ps1 (server name, credentials, output folder).
4. Run PowerShell as Administrator:
5. .\Run-HealthCheck.ps1
6. Results: per-check CSVs in output folder + HealthCheck_Summary.html.

1) T-SQL: HealthCheckQueries.sql

Save this as HealthCheckQueries.sql. It contains multiple named result sets; PowerShell will run each section and export to CSV/HTML.


```
/* HealthCheckQueries.sql
   Multiple sections separated by comments. Each section is intended to be run separately.
   Note: run under master context. */

-- INSTANCE_INFO
SELECT
   SERVERPROPERTY('MachineName') AS MachineName,
   SERVERPROPERTY('ServerName') AS ServerName,
   SERVERPROPERTY('InstanceName') AS InstanceName,
   SERVERPROPERTY('Edition') AS Edition,
   SERVERPROPERTY('ProductVersion') AS ProductVersion,
   SERVERPROPERTY('ProductLevel') AS ProductLevel,
   cpu_count = (SELECT cpu_count FROM sys.dm_os_sys_info),
   physical_memory_MB = (SELECT physical_memory_kb/1024 FROM sys.dm_os_sys_info);

-- SERVER_CONFIG (sp_configure snapshot)
EXEC sp_configure 'show advanced options', 1; RECONFIGURE;
EXEC sp_configure;

-- SERVICE_ACCOUNT
SELECT servicename = 'SQL Server', account = servicename = NULL; -- placeholder
-- Better collected from xp_cmdshell or PowerShell; PowerShell will fetch the service account.

-- MAX_MEMORY_AND_PARALLELISM
SELECT
```

```
    [name],
    [value],
    [value_in_use]
FROM sys.configurations
WHERE name IN ('max server memory (mb)', 'max degree of parallelism', 'cost threshold for parallelism');

-- TEMPDB_FILES
SELECT name, physical_name, size/128 AS SizeMB, max_size, growth
FROM tempdb.sys.database_files;

-- DATABASE_FILES
SELECT DB_NAME(database_id) AS DatabaseName, name, physical_name, size/128 AS SizeMB, max_size, growth
FROM sys.master_files
WHERE database_id > 4 -- exclude system DBs if desired
ORDER BY DatabaseName;

-- FILE_IO_STATS (per file)
SELECT
    DB_NAME(vfs.database_id) AS DatabaseName,
    vfs.file_id,
    mf.physical_name,
    total_io_stall_ms,
    num_of_reads, num_of_writes,
    CAST(total_io_stall_ms/(CASE WHEN num_of_reads=0 THEN 1.0 ELSE num_of_reads END) AS decimal(12,2)) AS
AvgReadLatency_ms,
    CAST(total_io_stall_ms/(CASE WHEN num_of_writes=0 THEN 1.0 ELSE num_of_writes END) AS decimal(12,2)) AS
AvgWriteLatency_ms
FROM sys.dm_io_virtual_file_stats(NULL,NULL) vfs
JOIN sys.master_files mf ON vfs.database_id = mf.database_id AND vfs.file_id = mf.file_id
ORDER BY AvgReadLatency_ms DESC;

-- PAGE_LIFE_EXPECTANCY
SELECT cntr_value AS PageLifeExpectancy
FROM sys.dm_os_performance_counters
WHERE counter_name = 'Page life expectancy';

-- MEMORY_CLERKS_TOP (top memory consumers)
SELECT TOP 20
    type,
    SUM(single_pages_kb + multi_pages_kb)/1024 AS MemoryMB
FROM sys.dm_os_memory_clerks
GROUP BY type
ORDER BY MemoryMB DESC;

-- CPU_TOP_QUERIES (top CPU consuming queries)
SELECT TOP 25
    qs.total_worker_time/1000 AS total_cpu_ms,
    qs.execution_count,
    qs.total_worker_time/qs.execution_count/1000.0 AS avg_cpu_ms,
    SUBSTRING(st.text, (qs.statement_start_offset/2)+1,
        ((CASE statement_end_offset WHEN -1 THEN DATALENGTH(st.text) ELSE qs.statement_end_offset END
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator


```
        - qs.statement_start_offset)/2)+1) AS statement_text,
    DB_NAME(st.dbid) AS DBName,
    qp.query_plan
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
CROSS APPLY sys.dm_exec_query_plan(qs.plan_handle) qp
ORDER BY qs.total_worker_time DESC;

-- WAIT_STATS (clean baseline)
SELECT TOP 50
    wait_type,
    wait_time_ms,
    waiting_tasks_count,
    signal_wait_time_ms
FROM sys.dm_os_wait_stats
WHERE wait_type NOT IN
('BROKER_TASK_STOP','BROKER_EVENTHANDLER','BROKER_RECEIVE_WAITFOR','BROKER_TRANSMITTER','CLR_SEMAPHORE','
LAZYWRITER_SLEEP','LOGMGR_QUEUE','CHECKPOINT_QUEUE','REQUEST_FOR_DEADLOCK_SEARCH','XE_TIMER_EVENT','XE_
DISPATCHER_JOIN','SQLTRACE_BUFFER_FLUSH')
ORDER BY wait_time_ms DESC;

-- MISSING_INDEXES (summary)
SELECT TOP 100
    migs.avg_user_impact,
    mid.statement AS database_schema_table,
    mid.equality_columns,
    mid.inequality_columns,
    mid.included_columns,
    migs.user_seeks,
    migs.user_scans,
    (migs.user_seeks + migs.user_scans) AS total_user_reads,
    migs.avg_total_user_cost
FROM sys.dm_db_missing_index_group_stats migs
JOIN sys.dm_db_missing_index_groups mig ON migs.group_handle = mig.index_group_handle
JOIN sys.dm_db_missing_index_details mid ON mig.index_handle = mid.index_handle
ORDER BY migs.avg_user_impact DESC, (migs.user_seeks + migs.user_scans) DESC;

-- INDEX_FRAGMENTATION (top fragmented indexes)
SELECT TOP 100
    DB_NAME(ps.database_id) AS DatabaseName,
    OBJECT_SCHEMA_NAME(ps.object_id, ps.database_id) AS SchemaName,
    OBJECT_NAME(ps.object_id, ps.database_id) AS ObjectName,
    i.name AS IndexName,
    ps.index_id,
    ps.avg_fragmentation_in_percent,
    ps.page_count
FROM sys.dm_db_index_physical_stats(NULL, NULL, NULL, NULL, 'LIMITED') ps
JOIN sys.indexes i ON ps.object_id = i.object_id AND ps.index_id = i.index_id
WHERE ps.database_id NOT IN (1,2,3,4) -- exclude master, tempdb, model, msdb
ORDER BY ps.avg_fragmentation_in_percent DESC;
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator


```
-- LAST_BACKUP (per database: last full & log)
SELECT
    d.name AS DatabaseName,
    MAX(CASE WHEN bs.type = 'D' THEN bs.backup_finish_date END) AS LastFullBackup,
    MAX(CASE WHEN bs.type = 'I' THEN bs.backup_finish_date END) AS LastDiffBackup,
    MAX(CASE WHEN bs.type = 'L' THEN bs.backup_finish_date END) AS LastLogBackup
FROM sys.databases d
LEFT JOIN msdb.dbo.backupset bs ON bs.database_name = d.name
GROUP BY d.name
ORDER BY d.name;
```

-- DBCC CHECKDB placeholder (run separately / scheduled)
-- Recommend running per database with NO_INFOMSGS and output to file.

2) PowerShell runner: Run-HealthCheck.ps1

Save as Run-HealthCheck.ps1. Edit at top to set server, credential method, and output path. This script will: split HealthCheckQueries.sql into named sections, run each query, export CSVs, and build a simple HTML summary.

```
<#
Run-HealthCheck.ps1
Usage: edit variables below and run from PowerShell (Run as Admin)
Prerequisites: Install-Module SqlServer
#>
```



```
param (
    [string]$SqlInstance = "localhost",      # e.g. 'myserver\SQL2019' or '127.0.0.1'
    [string]$OutputFolder = "C:\SqlHealthCheck\Output",
    [string]$SqlQueryFile = "C:\SqlHealthCheck\HealthCheckQueries.sql",
    [switch]$UseWindowsAuth = $true
)

# --- config
if (-not (Test-Path $OutputFolder)) { New-Item -Path $OutputFolder -ItemType Directory -Force | Out-Null }

Import-Module SqlServer -ErrorAction Stop

# credential
if ($UseWindowsAuth) {
    $Credential = $null
    Write-Host "Using Windows Authentication to connect to $SqlInstance"
} else {
    if (-not $Credential) {
        $Credential = Get-Credential -Message "Enter SQL Credentials"
    }
}

# helper: run named queries by splitting script at comment markers "-- SECTION_NAME"
$scriptText = Get-Content -Raw -Path $SqlQueryFile
# split by lines with -- followed by uppercase word at start
$sections = @()
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator


```
$currentName = "unnamed"
$currentSql = ""
foreach ($line in $scriptText -split "`r?`n") {
   if ($line -match '^\s*--\s*([A-Z0-9_]+)\s*$') {
      if ($currentSql.Trim()) {
         $sections += [PSCustomObject]@{ Name = $currentName; Sql = $currentSql }
      }
      $currentName = $Matches[1]
      $currentSql = ""
   } else {
      $currentSql += $line + "`r`n"
   }
}
# add last
if ($currentSql.Trim()) {
   $sections += [PSCustomObject]@{ Name = $currentName; Sql = $currentSql }
}

# execute each section and export CSV
$resultsMeta = @()
$sectionIndex = 0
foreach ($s in $sections) {
   $sectionIndex++
   $cleanName = ($s.Name).ToLower()
   $csvPath = Join-Path $OutputFolder ("{0:D2}_{1}.csv" -f $sectionIndex, $cleanName)
   Write-Host "Running section [$sectionIndex] $($s.Name) -> $csvPath"

   try {
      if ($UseWindowsAuth) {
         $data = Invoke-Sqlcmd -ServerInstance $SqlInstance -Query $s.Sql -ErrorAction Stop
      } else {
         $data = Invoke-Sqlcmd -ServerInstance $SqlInstance -Query $s.Sql -Username $Credential.UserName -Password
($Credential.GetNetworkCredential().Password) -ErrorAction Stop
      }

      if ($null -eq $data) {
         # create empty placeholder
         $table = @()
         $table | Export-Csv -Path $csvPath -NoTypeInformation -Force
      } else {
         $data | Export-Csv -Path $csvPath -NoTypeInformation -Force
      }
      $resultsMeta += [PSCustomObject]@{
         Section = $s.Name
         Rows = ($data | Measure-Object).Count
         Csv = $csvPath
      }
   } catch {
      Write-Warning "Failed section $($s.Name): $_"
      $resultsMeta += [PSCustomObject]@{
         Section = $s.Name
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator


```
            Rows = 0
            Csv = $csvPath
            Error = $_.Exception.Message
        }
    }
}

# collect additional info via PowerShell (service account, NIC, disks)
$svc = Get-Service -Name 'MSSQL*' -ErrorAction SilentlyContinue |
    Where-Object { $_.Status -ne $null } |
    Select-Object Name, DisplayName, Status

$svc | Export-Csv -Path (Join-Path $OutputFolder "Services.csv") -NoTypeInformation -Force

# NIC info
Get-NetAdapter | Select-Object Name, Status, LinkSpeed, ReceiveLinkSpeed, TransmitLinkSpeed |
    Export-Csv -Path (Join-Path $OutputFolder "NICs.csv") -NoTypeInformation -Force

# Disk info
Get-PSDrive -PSProvider FileSystem | Select-Object Name, Root, Used, Free |
    Export-Csv -Path (Join-Path $OutputFolder "Disks.csv") -NoTypeInformation -Force

# Build summary HTML
$htmlFile = Join-Path $OutputFolder "HealthCheck_Summary.html"
$now = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
$summaryRows = $resultsMeta | ForEach-Object {
    "<tr><td>$($_.Section)</td><td>$($_.Rows)</td><td><a href='file:///$($_.Csv)'>CSV</a></td><td>$($_.Error)</td></tr>"
} -join "`n"

$html = @"
<html>
<head>
 <meta charset='utf-8' />
 <title>SQL Health Check Summary - $SqlInstance</title>
 <style>
   body { font-family: Arial, sans-serif; padding: 20px; }
   table { border-collapse: collapse; width: 100%; }
   th, td { border: 1px solid #ddd; padding: 8px; }
   th { background: #f2f2f2; }
 </style>
</head>
<body>
 <h1>SQL Health Check Summary</h1>
 <p>Instance: <b>$SqlInstance</b></p>
 <p>Run Time: $now</p>
 <h2>Results</h2>
 <table>
   <thead><tr><th>Section</th><th>Rows</th><th>CSV</th><th>Error</th></tr></thead>
   <tbody>
   $summaryRows
   </tbody>
```

```
 </table>
 <p>Generated by Run-HealthCheck.ps1</p>
</body>
</html>
"@

Set-Content -Path $htmlFile -Value $html -Force -Encoding UTF8

Write-Host "All done. Outputs saved to $OutputFolder"
Write-Host "Open $htmlFile to view summary."
```

3) Optional: Run-DBCC-Checks.ps1 (separate, run off-hours)

DBCC CHECKDB is IO- and CPU-intensive. Save and run separately per database (or staggered).

```
param(
   [string]$SqlInstance = "localhost",
   [string]$OutputFolder = "C:\SqlHealthCheck\Output\DBCC"
)
if (-not (Test-Path $OutputFolder)) { New-Item -Path $OutputFolder -ItemType Directory -Force | Out-Null }
Import-Module SqlServer

# get databases to check (exclude offline, snapshot)
$databases = Invoke-Sqlcmd -ServerInstance $SqlInstance -Query "SELECT name FROM sys.databases WHERE
state_desc='ONLINE' AND database_id > 4"
```



```
foreach ($db in $databases) {
   $name = $db.name
   $logFile = Join-Path $OutputFolder ("DBCC_{0}_{1:yyyyMMdd_HHmm}.txt" -f $name, (Get-Date))
   Write-Host "Running DBCC CHECKDB for $name -> $logFile"
   try {
      Invoke-Sqlcmd -ServerInstance $SqlInstance -Database $name -Query "DBCC CHECKDB ([$name]) WITH
NO_INFOMSGS, ALL_ERRORMSGS;"
      # Note: redirect output to file by capturing results - or use sqlcmd with -o parameter for full messaging
   } catch {
      $_ | Out-File -FilePath $logFile -Append
   }
}
```

4) Example improvements & customizations you may want to add

- Add thresholds in PowerShell to flag: PLE < 300, AvgReadLatency > 10ms, AvgWriteLatency > 15ms, missing index impact > X, long-running queries > Y; then surface warnings in the HTML.
- Pull Azure VM metrics (CPU credits, disk metrics, NIC metrics) via Az PowerShell / Get-AzMetric.
- Integrate with Log Analytics: push CSVs or send summary to an Azure Monitor workspace.
- Add email delivery: use Send-MailMessage or Graph API to send the HTML summary.
- Replace CSV links with embedded HTML tables for a richer single-file report.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

5) Quick recommended checks to run immediately after health-check

1. Verify TempDB file count and equal sizes.
2. Check expensive waits (PAGEIOLATCH, LCK_M, CXPACKET) and correlate with top queries.
3. Inspect top IO-latency files and map to disk SKU (Premium/Ultra).
4. Validate backups: last full/log times and restore test for at least one DB.
5. Review Azure Advisor for VM resizing or Reserved Instance opportunities.

Add threshold-based **warning badges** in the HTML (e.g., Red/Yellow/Green) and auto-highlighting for problem rows.

Let's enhance the **Run-HealthCheck.ps1** script to include **threshold-based warning badges** and **auto-highlighting** for key metrics — so you can visually spot performance or configuration issues right in the HTML summary.
Below is the **improved version** of the script with:

- ✅ Red / Yellow / Green status indicators (Bad / Warning / Healthy)
- ✅ Auto-analysis of CSV results (Page Life Expectancy, IO latency, waits, backups, etc.)
- ✅ Inline HTML badges and row highlighting
- ✅ A compact, professional summary report

⚡ **Updated PowerShell Script — Run-HealthCheck.ps1**

```
<#
Enhanced SQL Health Check Runner
Adds threshold-based color badges (Red/Yellow/Green)
Author: Praveen + Sage
#>

param (
   [string]$SqlInstance = "localhost",
   [string]$OutputFolder = "C:\SqlHealthCheck\Output",
   [string]$SqlQueryFile = "C:\SqlHealthCheck\HealthCheckQueries.sql",
   [switch]$UseWindowsAuth = $true
)

if (-not (Test-Path $OutputFolder)) { New-Item -Path $OutputFolder -ItemType Directory -Force | Out-Null }
Import-Module SqlServer -ErrorAction Stop

if ($UseWindowsAuth) {
   $Credential = $null
   Write-Host "Using Windows Authentication to connect to $SqlInstance"
} else {
   if (-not $Credential) { $Credential = Get-Credential -Message "Enter SQL Credentials" }
}

# --- Split T-SQL script into sections
$scriptText = Get-Content -Raw -Path $SqlQueryFile
$sections = @()
$currentName = "unnamed"
$currentSql = ""
foreach ($line in $scriptText -split "`r?`n") {
```

```
    if ($line -match '^\s*--\s*([A-Z0-9_]+)\s*$') {
        if ($currentSql.Trim()) { $sections += [PSCustomObject]@{ Name = $currentName; Sql = $currentSql } }
        $currentName = $Matches[1]
        $currentSql = ""
    } else {
        $currentSql += $line + "`r`n"
    }
}
if ($currentSql.Trim()) { $sections += [PSCustomObject]@{ Name = $currentName; Sql = $currentSql } }

$resultsMeta = @()
$sectionIndex = 0
foreach ($s in $sections) {
    $sectionIndex++
    $cleanName = ($s.Name).ToLower()
    $csvPath = Join-Path $OutputFolder ("{0:D2}_{1}.csv" -f $sectionIndex, $cleanName)
    Write-Host "Running [$sectionIndex] $($s.Name)..."
    try {
        if ($UseWindowsAuth) {
            $data = Invoke-Sqlcmd -ServerInstance $SqlInstance -Query $s.Sql -ErrorAction Stop
        } else {
            $data = Invoke-Sqlcmd -ServerInstance $SqlInstance -Query $s.Sql -Username $Credential.UserName -Password
($Credential.GetNetworkCredential().Password) -ErrorAction Stop
        }
        if ($null -eq $data) { @() | Export-Csv -Path $csvPath -NoTypeInformation -Force }
        else { $data | Export-Csv -Path $csvPath -NoTypeInformation -Force }
        $resultsMeta += [PSCustomObject]@{ Section = $s.Name; Csv = $csvPath; Rows = ($data | Measure-Object).Count; Error
= "" }
    } catch {
        Write-Warning "Error running $($s.Name): $_"
        $resultsMeta += [PSCustomObject]@{ Section = $s.Name; Csv = $csvPath; Rows = 0; Error = $_.Exception.Message }
    }
}

# --- Add environment checks (service, NIC, disks)
Get-Service -Name 'MSSQL*' | Select Name,DisplayName,Status | Export-Csv -Path (Join-Path $OutputFolder "Services.csv") -
NoTypeInformation
Get-NetAdapter | Select Name,Status,LinkSpeed | Export-Csv -Path (Join-Path $OutputFolder "NICs.csv") -NoTypeInformation
Get-PSDrive -PSProvider FileSystem | Select Name,Root,Used,Free | Export-Csv -Path (Join-Path $OutputFolder "Disks.csv") -
NoTypeInformation

# --- Analysis thresholds
$badges = @()
function Get-Badge($value, $good, $warn) {
    if ($value -lt $good) { return "<span class='badge red'>Critical</span>" }
    elseif ($value -lt $warn) { return "<span class='badge yellow'>Warning</span>" }
    else { return "<span class='badge green'>Healthy</span>" }
}

# PAGE LIFE EXPECTANCY (PLE)
$pleCsv = (Get-ChildItem $OutputFolder | Where-Object Name -like "*page_life_expectancy*.csv").FullName
```

```
if ($pleCsv) {
   $pleVal = (Import-Csv $pleCsv | Select-Object -First 1).PageLifeExpectancy
   if ($pleVal) {
      $pleBadge = Get-Badge $pleVal 300 600
      $badges += "<tr><td>Page Life Expectancy</td><td>$pleVal</td><td>$pleBadge</td><td>Target >600</td></tr>"
   }
}

# IO LATENCY
$ioCsv = (Get-ChildItem $OutputFolder | Where-Object Name -like "*file_io_stats*.csv").FullName
if ($ioCsv) {
   $ioData = Import-Csv $ioCsv | Sort-Object {[double]$_.AvgReadLatency_ms} -Descending | Select -First 3
   foreach ($row in $ioData) {
      $r = [double]$row.AvgReadLatency_ms
      $badge = Get-Badge (1000/$r) 100 50 # invert logic
      $badges += "<tr><td>IO Latency (Read)</td><td>$($row.physical_name): $r ms</td><td>$badge</td><td><5ms
ideal</td></tr>"
   }
}

# WAIT STATS
$waitCsv = (Get-ChildItem $OutputFolder | Where-Object Name -like "*wait_stats*.csv").FullName
if ($waitCsv) {
   $waitData = Import-Csv $waitCsv | Select-Object -First 10
   foreach ($w in $waitData) {
      $badge = if ($w.wait_type -match "CXPACKET|PAGEIOLATCH|LCK_M") { "<span class='badge yellow'>Investigate</span>"
} else { "<span class='badge green'>OK</span>" }
      $badges += "<tr><td>Wait:
$($w.wait_type)</td><td>$([math]::Round($w.wait_time_ms/1000,1))s</td><td>$badge</td><td>Check top
waits</td></tr>"
   }
}

# BACKUPS
$backupCsv = (Get-ChildItem $OutputFolder | Where-Object Name -like "*last_backup*.csv").FullName
if ($backupCsv) {
   $backups = Import-Csv $backupCsv
   foreach ($b in $backups) {
      $days = if ($b.LastFullBackup) { (New-TimeSpan -Start $b.LastFullBackup -End (Get-Date)).Days } else { 999 }
      $badge = Get-Badge (1000/$days) 100 50
      $badges += "<tr><td>Backup - $($b.DatabaseName)</td><td>$days days ago</td><td>$badge</td><td>Full backup < 7
days old</td></tr>"
   }
}

# --- Build HTML summary
$htmlFile = Join-Path $OutputFolder "HealthCheck_Summary.html"
$now = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
$summaryRows = $resultsMeta | ForEach-Object {
   $color = if ($_.Error) {"style='background:#ffcccc'"} else {"style='background:#eaffea'"}
```

```
    "<tr $color><td>$($_.Section)</td><td>$($_.Rows)</td><td><a
href='file:///$($_.Csv)'>CSV</a></td><td>$($_.Error)</td></tr>"
} -join "`n"

$analysisTable = if ($badges.Count -gt 0) {
    "<table><thead><tr><th>Metric</th><th>Value</th><th>Status</th><th>Target</th></tr></thead><tbody>$($badges -
join "`n")</tbody></table>"
} else {
    "<p>No analysis metrics computed.</p>"
}

$html = @"
<html>
<head>
<meta charset='utf-8'>
<title>SQL Health Check Report - $SqlInstance</title>
<style>
body { font-family: Segoe UI, sans-serif; padding: 20px; }
h1 { color: #0078D4; }
table { border-collapse: collapse; width: 100%; margin-bottom: 20px; }
th, td { border: 1px solid #ccc; padding: 8px; text-align: left; }
th { background-color: #f3f3f3; }
.badge { padding: 4px 8px; border-radius: 6px; color: white; font-weight: bold; }
.red { background-color: #e74c3c; }
.yellow { background-color: #f1c40f; color: #000; }
.green { background-color: #2ecc71; }
</style>
</head>
<body>
<h1>SQL Server Health Check Report</h1>
<p><b>Instance:</b> $SqlInstance</p>
<p><b>Generated:</b> $now</p>

<h2>Automated Analysis (Thresholds)</h2>
$analysisTable

<h2>Section Summary</h2>
<table>
<thead><tr><th>Section</th><th>Rows</th><th>CSV Link</th><th>Error</th></tr></thead>
<tbody>
$summaryRows
</tbody>
</table>

<p style='font-size:12px;color:#777'>Generated automatically by Run-HealthCheck.ps1 (Sage Edition)</p>
</body></html>
"@

Set-Content -Path $htmlFile -Value $html -Encoding UTF8 -Force
```

```
Write-Host "✅ Health Check completed."
Write-Host "Report: $htmlFile"
```

🟢 What's New

Metric	Threshold Logic	Badge Meaning
Page Life Expectancy (PLE)	<300 → 🔴 Critical, <600 → 🟡 Warning, ≥600 → 🟢 Healthy	Memory pressure check
IO Latency (ms)	>10ms → 🔴 Critical, 5–10ms → 🟡 Warning, <5ms → 🟢 Healthy	Disk performance
Wait Stats	Common blocking waits (CXPACKET, PAGEIOLATCH, LCK_M) → 🟡 Investigate	CPU, IO, lock waits
Backups	Full backup older than 7 days → 🔴, 4–7 days → 🟡	Backup recency check
Errors per section	Any query failure = highlighted row in pink	Execution issue

📈 Result

When you open HealthCheck_Summary.html, you'll see:

- A **color-coded dashboard** with badges and thresholds.
- A **clickable summary table** linking to each section's CSV file.
- A clean, client-ready format — ideal for Azure VM SQL health reports.

"HTML Detailed Report" version — where each section's data (like TempDB, Wait Stats, IO, Backups, etc.) is **embedded directly inside the HTML** with collapsible panels (no external CSVs needed). That's perfect for single-file delivery to clients or management.

Before we generate **automated SQL Server on Azure VM Health Check HTML Report Pack**, let's finalize the structure so that we can include proper threshold-based color coding and alerts.
Here's what we'll include — please confirm or edit any of these:

🧩 Report Sections

1. **Instance Overview**
   - SQL Version, Edition, Build
   - Uptime
   - CPU, Memory (Total/Used)
   - Database Count

     🔷 **Badge logic:**
   - Memory usage >85% → 🔴 Red
   - Memory usage 70–85% → 🟡 Yellow
   - <70% → 🟢 Green

2. **Configuration Check**
   - Max Server Memory
   - MAXDOP
   - Cost Threshold for Parallelism
   - TempDB Files count & size

🔷**Badge logic:**

- MAXDOP > 8 or = 0 → 🔴
- Cost Threshold < 25 → 🟡
- TempDB files < (logical CPUs / 4) → 🟡

3. **Performance Metrics**
   - Top 10 CPU Queries
   - Top 10 IO Queries
   - Wait Statistics summary

   🔷**Badge logic:**
   - High CXPACKET or PAGEIOLATCH waits → 🔴
   - Moderate LCK_M waits → 🟡

4. **Storage & IO Health**
   - Disk latency (Read/Write)
   - File growth settings

   🔷**Badge logic:**
   - Read/Write latency > 15 ms → 🔴
   - 5–15 ms → 🟡
   - < 5 ms → 🟢

5. **TempDB & Log Space**
   - TempDB free space %
   - Log file usage %

   🔷**Badge logic:**
   - Free space < 10% → 🔴
   - 10–25% → 🟡

6. **Backup & Job Status**
   - Last backup time per DB
   - Job failures in last 24 hours

   🔷**Badge logic:**
   - Missing full backup >24 hrs → 🔴
   - Failed job count >0 → 🔴

7. **Recommendations Summary**
   - Color-coded summary with problem areas auto-highlighted.

⚙ **Output**

- SQL_Azure_HealthCheck_Report.html
- Includes embedded **CSS badges (🟢🟡🔴)** and JavaScript auto-highlighting.
- Generated using a **PowerShell script** that runs T-SQL checks and outputs to HTML.

Ready-to-run T-SQL + PowerShell combo that automates the full health check for **SQL Server on an Azure VM**, evaluates thresholds, and produces a **single self-contained HTML report** with colored badges (Red/Yellow/Green), collapsible sections, and auto-highlighting of problem rows.

What you get right now in this message

- HealthCheckQueries.sql — the T-SQL file with named sections (PowerShell will split by -- SECTION_NAME comments).
- Run-HealthCheck-FullReport.ps1 — PowerShell runner that:
  - Executes each T-SQL section against the instance.
  - Collects results, applies threshold logic.
  - Embeds all results into a single SQL_Azure_HealthCheck_Report.html (no external CSVs).
  - Adds badges, collapsible panels, and a summary recommendations block.

No additional questions — copy the files into a folder (e.g., C:\SqlHealthCheck) and run the PowerShell script.

1) HealthCheckQueries.sql

Save as C:\SqlHealthCheck\HealthCheckQueries.sql

```
/* HealthCheckQueries.sql
  Designed to be split by the PowerShell runner using '-- SECTION_NAME' markers.
  Run under master context or a user with access to DMVs and msdb.
*/

-- INSTANCE_OVERVIEW
SELECT
  SERVERPROPERTY('MachineName') AS MachineName,
  SERVERPROPERTY('ServerName') AS ServerName,
  SERVERPROPERTY('InstanceName') AS InstanceName,
  SERVERPROPERTY('Edition') AS Edition,
  SERVERPROPERTY('ProductVersion') AS ProductVersion,
  SERVERPROPERTY('ProductLevel') AS ProductLevel,
  sql_start_time = sqlserver_start_time,
  cpu_count = (SELECT cpu_count FROM sys.dm_os_sys_info),
  physical_memory_MB = (SELECT physical_memory_kb/1024 FROM sys.dm_os_sys_info),
  DatabaseCount = (SELECT count(*) FROM sys.databases)
FROM sys.dm_os_sys_info AS s
CROSS APPLY (SELECT sqlserver_start_time = sqlserver_start_time FROM sys.dm_os_sys_info) AS ca;

-- UPTIME
SELECT sqlserver_start_time, DATEDIFF(SECOND, sqlserver_start_time, GETDATE()) AS SecondsUp,
    CONVERT(varchar(50), DATEADD(SECOND, DATEDIFF(SECOND, sqlserver_start_time, GETDATE()), 0), 120) AS UptimeApprox
FROM sys.dm_os_sys_info;

-- CONFIGURATION
SELECT
  name, value, value_in_use, description
FROM sys.configurations
WHERE name IN ('max server memory (mb)', 'max degree of parallelism', 'cost threshold for parallelism');
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator


```
-- TEMPDB_FILES
SELECT name, physical_name, size/128 AS SizeMB, max_size, growth
FROM tempdb.sys.database_files;

-- TEMPDB_SPACE
SELECT
   SUM(size) * 8.0/1024 AS TempDB_TotalMB,
   SUM(CAST(FILEPROPERTY(name,'SpaceUsed') AS INT)) * 8.0/1024 AS TempDB_UsedMB,
   (SUM(size) - SUM(CAST(FILEPROPERTY(name,'SpaceUsed') AS INT))) * 8.0/1024 AS TempDB_FreeMB
FROM tempdb.sys.database_files;

-- DATABASE_FILE_USAGE
SELECT DB_NAME(mf.database_id) AS DatabaseName, mf.name, mf.physical_name,
     mf.size/128 AS SizeMB, mf.max_size, mf.growth,
     CAST(100.0 * FILEPROPERTY(mf.name,'SpaceUsed') / NULLIF(mf.size,0) AS DECIMAL(6,2)) AS PercentUsed
FROM sys.master_files mf
ORDER BY DB_NAME(mf.database_id), mf.file_id;

-- FILE_IO_STATS
SELECT
   DB_NAME(vfs.database_id) AS DatabaseName,
   vfs.file_id,
   mf.physical_name,
   vfs.num_of_reads,
   vfs.num_of_writes,
   CASE WHEN vfs.num_of_reads=0 THEN 0 ELSE CAST(vfs.io_stall_read_ms/NULLIF(vfs.num_of_reads,0) AS DECIMAL(12,2)) END
AS AvgReadLatency_ms,
   CASE WHEN vfs.num_of_writes=0 THEN 0 ELSE CAST(vfs.io_stall_write_ms/NULLIF(vfs.num_of_writes,0) AS DECIMAL(12,2))
END AS AvgWriteLatency_ms
FROM sys.dm_io_virtual_file_stats(NULL,NULL) vfs
JOIN sys.master_files mf ON vfs.database_id = mf.database_id AND vfs.file_id = mf.file_id
ORDER BY AvgReadLatency_ms DESC;

-- PAGE_LIFE_EXPECTANCY
SELECT cntr_value AS PageLifeExpectancy
FROM sys.dm_os_performance_counters
WHERE counter_name = 'Page life expectancy';

-- MEMORY_CLERKS_TOP
SELECT TOP 20
   type,
   (SUM(single_pages_kb + multi_pages_kb)/1024) AS MemoryMB
FROM sys.dm_os_memory_clerks
GROUP BY type
ORDER BY MemoryMB DESC;
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator


```
-- TOP_CPU_QUERIES
SELECT TOP 20
   qs.total_worker_time/1000 AS total_cpu_ms,
   qs.execution_count,
   (qs.total_worker_time/NULLIF(qs.execution_count,1))/1000.0 AS avg_cpu_ms,
   SUBSTRING(st.text, (qs.statement_start_offset/2)+1,
      ((CASE qs.statement_end_offset WHEN -1 THEN DATALENGTH(st.text) ELSE qs.statement_end_offset END
       - qs.statement_start_offset)/2)+1) AS statement_text,
   DB_NAME(st.dbid) AS DBName
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
ORDER BY qs.total_worker_time DESC;

-- TOP_IO_QUERIES
SELECT TOP 20
   qs.total_logical_reads AS total_logical_reads,
   qs.total_physical_reads AS total_physical_reads,
   qs.total_logical_reads/NULLIF(qs.execution_count,1) AS avg_logical_reads,
   SUBSTRING(st.text, (qs.statement_start_offset/2)+1,
      ((CASE qs.statement_end_offset WHEN -1 THEN DATALENGTH(st.text) ELSE qs.statement_end_offset END
       - qs.statement_start_offset)/2)+1) AS statement_text,
   DB_NAME(st.dbid) AS DBName
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
ORDER BY qs.total_logical_reads DESC;
```



```
-- WAIT_STATS
SELECT TOP 50
   wait_type,
   wait_time_ms,
   waiting_tasks_count,
   signal_wait_time_ms
FROM sys.dm_os_wait_stats
WHERE wait_type NOT IN
('BROKER_TASK_STOP','BROKER_EVENTHANDLER','BROKER_RECEIVE_WAITFOR','BROKER_TRANSMITTER','CLR_SEMAPHORE','LAZ
YWRITER_SLEEP','LOGMGR_QUEUE','CHECKPOINT_QUEUE','REQUEST_FOR_DEADLOCK_SEARCH','XE_TIMER_EVENT','XE_DISPATC
HER_JOIN','SQLTRACE_BUFFER_FLUSH')
ORDER BY wait_time_ms DESC;

-- MISSING_INDEXES
SELECT TOP 100
   migs.avg_user_impact,
   mid.statement AS database_schema_table,
   mid.equality_columns,
   mid.inequality_columns,
   mid.included_columns,
   migs.user_seeks,
   migs.user_scans,
   (migs.user_seeks + migs.user_scans) AS total_user_reads
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator


```
FROM sys.dm_db_missing_index_group_stats migs
JOIN sys.dm_db_missing_index_groups mig ON migs.group_handle = mig.index_group_handle
JOIN sys.dm_db_missing_index_details mid ON mig.index_handle = mid.index_handle
ORDER BY migs.avg_user_impact DESC, (migs.user_seeks + migs.user_scans) DESC;

-- INDEX_FRAGMENTATION
SELECT TOP 200
   DB_NAME(ps.database_id) AS DatabaseName,
   OBJECT_SCHEMA_NAME(ps.object_id, ps.database_id) AS SchemaName,
   OBJECT_NAME(ps.object_id, ps.database_id) AS ObjectName,
   i.name AS IndexName,
   ps.index_id,
   ps.avg_fragmentation_in_percent,
   ps.page_count
FROM sys.dm_db_index_physical_stats(NULL, NULL, NULL, NULL, 'LIMITED') ps
JOIN sys.indexes i ON ps.object_id = i.object_id AND ps.index_id = i.index_id
WHERE ps.database_id > 4
ORDER BY ps.avg_fragmentation_in_percent DESC;

-- LAST_BACKUP
SELECT
   d.name AS DatabaseName,
   MAX(CASE WHEN bs.type = 'D' THEN bs.backup_finish_date END) AS LastFullBackup,
   MAX(CASE WHEN bs.type = 'I' THEN bs.backup_finish_date END) AS LastDiffBackup,
   MAX(CASE WHEN bs.type = 'L' THEN bs.backup_finish_date END) AS LastLogBackup
FROM sys.databases d
LEFT JOIN msdb.dbo.backupset bs ON bs.database_name = d.name
GROUP BY d.name
ORDER BY d.name;

-- AGENT_JOB_FAILURES_24H
SELECT
   j.name AS JobName,
   h.step_name,
   h.run_status,
   h.run_date,
   h.run_time,
   h.message
FROM msdb.dbo.sysjobhistory h
JOIN msdb.dbo.sysjobs j ON h.job_id = j.job_id
WHERE h.run_date >= CONVERT(INT, CONVERT(VARCHAR(8), DATEADD(DAY, -1, GETDATE()),112)) AND h.step_id = 0 AND
h.run_status <> 1
ORDER BY h.run_date DESC, h.run_time DESC;

-- LOGFILE_USAGE
SELECT
   DB_NAME(database_id) AS DatabaseName,
   type_desc,
   name,
```

```
    size/128 AS SizeMB,
    CAST(FILEPROPERTY(name,'SpaceUsed') AS INT)*8.0/1024 AS UsedMB,
    CAST(100.0 * FILEPROPERTY(name,'SpaceUsed') / NULLIF(size,0) AS DECIMAL(6,2)) AS PercentUsed
FROM sys.master_files
WHERE type_desc = 'LOG'
ORDER BY PercentUsed DESC;

-- QUICK_SUMMARY_METRICS
SELECT
    (SELECT cntr_value FROM sys.dm_os_performance_counters WHERE counter_name='Page life expectancy') AS
PageLifeExpectancy,
    (SELECT SUM(size)*8.0/1024 FROM tempdb.sys.database_files) AS TempDB_TotalMB,
    (SELECT SUM(CAST(FILEPROPERTY(name,'SpaceUsed') AS INT))*8.0/1024 FROM tempdb.sys.database_files) AS
TempDB_UsedMB,
    (SELECT cpu_count FROM sys.dm_os_sys_info) AS CPU_Count,
    (SELECT physical_memory_kb/1024 FROM sys.dm_os_sys_info) AS PhysicalMemoryMB;
```

2) Run-HealthCheck-FullReport.ps1

Save as C:\SqlHealthCheck\Run-HealthCheck-FullReport.ps1

Then run from PowerShell (Run as Administrator) after editing the parameters at top.

```
<#
Run-HealthCheck-FullReport.ps1
Executes HealthCheckQueries.sql and produces SQL_Azure_HealthCheck_Report.html
Prerequisites:
 - PowerShell (5.1+ or PowerShell Core)
 - SqlServer module: Install-Module SqlServer -Scope CurrentUser
 - Run as a user with permissions to query DMVs and msdb
Usage:
 .\Run-HealthCheck-FullReport.ps1 -SqlInstance "localhost" -OutputFolder "C:\SqlHealthCheck\Output" -UseWindowsAuth
#>

param (
    [string]$SqlInstance = "localhost",
    [string]$SqlQueryFile = "C:\SqlHealthCheck\HealthCheckQueries.sql",
    [string]$OutputFolder = "C:\SqlHealthCheck\Output",
    [switch]$UseWindowsAuth = $true
)

Set-StrictMode -Version Latest
if (-not (Test-Path $OutputFolder)) { New-Item -Path $OutputFolder -ItemType Directory -Force | Out-Null }
Import-Module SqlServer -ErrorAction Stop

Write-Host "Connecting to $SqlInstance ..."
# Credential handling
if ($UseWindowsAuth) {
    $SqlCredential = $null
} else {
    $SqlCredential = Get-Credential -Message "Enter SQL credentials"
}
```

```
# -------------------------
# Read and split SQL sections
# -------------------------
$scriptText = Get-Content -Raw -Path $SqlQueryFile -ErrorAction Stop
$sections = @()
$currentName = "unnamed"
$currentSql = ""
foreach ($line in $scriptText -split "`r?`n") {
   if ($line -match '^\s*--\s*([A-Z0-9_]+)\s*$') {
      if ($currentSql.Trim()) { $sections += [PSCustomObject]@{ Name = $currentName; Sql = $currentSql } }
      $currentName = $Matches[1]
      $currentSql = ""
   } else {
      $currentSql += $line + "`r`n"
   }
}
if ($currentSql.Trim()) { $sections += [PSCustomObject]@{ Name = $currentName; Sql = $currentSql } }

# -------------------------
# Run each section and store in memory
# -------------------------
$reports = @{}
foreach ($s in $sections) {
   Write-Host "Running section: $($s.Name)"
   try {
      if ($UseWindowsAuth) {
         $dt = Invoke-Sqlcmd -ServerInstance $SqlInstance -Query $s.Sql -ErrorAction Stop
      } else {
         $dt = Invoke-Sqlcmd -ServerInstance $SqlInstance -Query $s.Sql -Username $SqlCredential.UserName -Password
($SqlCredential.GetNetworkCredential().Password) -ErrorAction Stop
      }
      # Keep DataTable or array
      $reports[$s.Name] = $dt
   } catch {
      Write-Warning "Section $($s.Name) failed: $_"
      $reports[$s.Name] = @()  # empty
      $reports["__Errors__"] += "$($s.Name): $($_.Exception.Message)`n"
   }
}

# -------------------------
# Helper: Convert object/array to HTML fragment
# -------------------------
function To-SectionHtml([string]$sectionTitle, $data, [string]$id) {
   $html = "<div class='panel'><button class='collapsible' onclick=""togglePanel('$id')"" aria-
expanded='false'>$sectionTitle</button><div id='$id' class='content'>"
   if ($data -and ($data | Measure-Object).Count -gt 0) {
      # Use ConvertTo-Html for table body, then strip html wrapper
      $tbl = $data | ConvertTo-Html -Fragment -PreContent "" -PostContent "" -As Table -Property * 2>$null
      $html += $tbl
   } else {
```

```
        $html += "<p><i>No rows returned or query failed.</i></p>"
    }
    $html += "</div></div>"
    return $html
}

# -------------------------
# Extract key metric values for badges (use QUICK_SUMMARY_METRICS, PAGE_LIFE_EXPECTANCY, FILE_IO_STATS,
LAST_BACKUP, LOGFILE_USAGE)
# -------------------------
# Default placeholders
$metrics = @{
    PageLifeExpectancy = $null
    TempDB_TotalMB = $null
    TempDB_UsedMB = $null
    CPU_Count = $null
    PhysicalMemoryMB = $null
    TopIOReadMs = $null
    TopIOWriteMs = $null
    MissingFullBackups = @()
    LogFileHighUsage = @()
    HighWaits = @()
    TempDB_FreePct = $null
}



# Quick summary
if ($reports.ContainsKey('QUICK_SUMMARY_METRICS') -and $reports['QUICK_SUMMARY_METRICS']) {
    $row = $reports['QUICK_SUMMARY_METRICS'] | Select-Object -First 1
    $metrics.PageLifeExpectancy = [int]$row.PageLifeExpectancy
    $metrics.TempDB_TotalMB = [double]$row.TempDB_TotalMB
    $metrics.TempDB_UsedMB = [double]$row.TempDB_UsedMB
    if ($metrics.TempDB_TotalMB -and $metrics.TempDB_TotalMB -gt 0) {
        $metrics.TempDB_FreePct = ([math]::Round((($metrics.TempDB_TotalMB -
$metrics.TempDB_UsedMB)/$metrics.TempDB_TotalMB)*100,2))
    }
    $metrics.CPU_Count = [int]$row.CPU_Count
    $metrics.PhysicalMemoryMB = [double]$row.PhysicalMemoryMB
}

# File IO stats -> pick top read/write
if ($reports.ContainsKey('FILE_IO_STATS') -and $reports['FILE_IO_STATS']) {
    $io = $reports['FILE_IO_STATS'] | Sort-Object {[double]$_.AvgReadLatency_ms} -Descending | Select-Object -First 1
    if ($io) { $metrics.TopIOReadMs = [double]$io.AvgReadLatency_ms }
    $ioW = $reports['FILE_IO_STATS'] | Sort-Object {[double]$_.AvgWriteLatency_ms} -Descending | Select-Object -First 1
    if ($ioW) { $metrics.TopIOWriteMs = [double]$ioW.AvgWriteLatency_ms }
}

# Last backups -> gather DBs with older backups (>7 days)
if ($reports.ContainsKey('LAST_BACKUP') -and $reports['LAST_BACKUP']) {
    foreach ($b in $reports['LAST_BACKUP']) {
        $db = $b.DatabaseName
```

```
        $lastFull = $b.LastFullBackup
        if (-not $lastFull) {
            $metrics.MissingFullBackups += [PSCustomObject]@{ Database = $db; DaysSinceFull = 9999 }
        } else {
            $days = (New-TimeSpan -Start $lastFull -End (Get-Date)).TotalDays
            if ($days -gt 7) {
                $metrics.MissingFullBackups += [PSCustomObject]@{ Database = $db; DaysSinceFull = [math]::Round($days,1) }
            }
        }
    }
}

# Log file usage -> collect >80%
if ($reports.ContainsKey('LOGFILE_USAGE') -and $reports['LOGFILE_USAGE']) {
    foreach ($l in $reports['LOGFILE_USAGE']) {
        $pct = [double]$l.PercentUsed
        if ($pct -ge 80) {
            $metrics.LogFileHighUsage += [PSCustomObject]@{ Database = $l.DatabaseName; Percent = $pct }
        }
    }
}

# Wait stats -> check for known problematic wait types on top list
if ($reports.ContainsKey('WAIT_STATS') -and $reports['WAIT_STATS']) {
    $topWaits = $reports['WAIT_STATS'] | Select-Object -First 10
    foreach ($w in $topWaits) {
        if ($w.wait_type -match 'PAGEIOLATCH|LCK_M|CXPACKET|ASYNC_NETWORK_IO') {
            $metrics.HighWaits += $w.wait_type
        }
    }
}

# -------------------------
# Threshold helpers and badge generator
# -------------------------
function BadgeHtml([string]$status) {
    switch ($status) {
        'Critical' { return "<span class='badge red'>Critical</span>" }
        'Warning'  { return "<span class='badge yellow'>Warning</span>" }
        'Healthy'  { return "<span class='badge green'>Healthy</span>" }
        default    { return "<span class='badge grey'>Unknown</span>" }
    }
}

function Rate-PLE($val) {
    if ($null -eq $val) { return 'Unknown' }
    if ($val -lt 300) { return 'Critical' }
    if ($val -lt 600) { return 'Warning' }
    return 'Healthy'
}
function Rate-IO($ms) {
```

```
    if ($null -eq $ms) { return 'Unknown' }
    if ($ms -ge 15) { return 'Critical' }
    if ($ms -ge 5) { return 'Warning' }
    return 'Healthy'
}
function Rate-MemoryUsage($usedMB, $totalMB) {
    if (($totalMB -eq 0) -or ($totalMB -eq $null)) { return 'Unknown' }
    $pct = ($usedMB / $totalMB) * 100
    if ($pct -ge 85) { return 'Critical' }
    if ($pct -ge 70) { return 'Warning' }
    return 'Healthy'
}

# Build overall analysis table rows
$analysisRows = @()
# PLE
$pleStatus = Rate-PLE $metrics.PageLifeExpectancy
$analysisRows += [PSCustomObject]@{ Metric='Page Life Expectancy (sec)'; Value = $metrics.PageLifeExpectancy; Status =
$pleStatus; Target = '>=600 (Good)' }

# IO Read
$ioRStatus = Rate-IO $metrics.TopIOReadMs
$analysisRows += [PSCustomObject]@{ Metric='Top Read Latency (ms)'; Value = $metrics.TopIOReadMs; Status = $ioRStatus;
Target = '<5ms ideal' }

# IO Write
$ioWStatus = Rate-IO $metrics.TopIOWriteMs
$analysisRows += [PSCustomObject]@{ Metric='Top Write Latency (ms)'; Value = $metrics.TopIOWriteMs; Status =
$ioWStatus; Target = '<5ms ideal' }

# TempDB free
if ($metrics.TempDB_FreePct -ne $null) {
    $tmpStatus = if ($metrics.TempDB_FreePct -lt 10) {'Critical'} elseif ($metrics.TempDB_FreePct -lt 25) {'Warning'} else
{'Healthy'}
    $analysisRows += [PSCustomObject]@{ Metric='TempDB Free %'; Value = "$($metrics.TempDB_FreePct) %"; Status =
$tmpStatus; Target = '>25% free' }
}

# Log file usage
if ($metrics.LogFileHighUsage.Count -gt 0) {
    foreach ($l in $metrics.LogFileHighUsage) {
        $analysisRows += [PSCustomObject]@{ Metric = "Log Usage - $($l.Database)"; Value = "$($l.Percent) %"; Status =
'Critical'; Target = '<80%' }
    }
}

# Backups
if ($metrics.MissingFullBackups.Count -gt 0) {
    foreach ($m in $metrics.MissingFullBackups) {
        $s = if ($m.DaysSinceFull -ge 999) {'Critical'} elseif ($m.DaysSinceFull -gt 7) {'Critical'} else {'Warning'}
```

```
        $analysisRows += [PSCustomObject]@{ Metric = "LastFullBackup - $($m.Database)"; Value = "$($m.DaysSinceFull) days";
Status = $s; Target = '<=7 days' }
    }
} else {
    $analysisRows += [PSCustomObject]@{ Metric = "Backups"; Value = "All DBs have recent full backups"; Status = 'Healthy';
Target = '<=7 days' }
}

# Waits
if ($metrics.HighWaits.Count -gt 0) {
    $uniqueWaits = $metrics.HighWaits | Select-Object -Unique
    foreach ($w in $uniqueWaits) {
        $analysisRows += [PSCustomObject]@{ Metric = "High Wait - $w"; Value = ""; Status = 'Warning'; Target = 'Investigate top
queries' }
    }
}

# -------------------------
# Build HTML (embedded CSS/JS)
# -------------------------
$reportTitle = "SQL Azure VM Health Check Report - $SqlInstance"
$htmlPath = Join-Path $OutputFolder "SQL_Azure_HealthCheck_Report.html"

$style = @"
<style>
body { font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Arial; margin: 18px; color: #222; }
h1 { color: #0b5fff; }
.badge { padding: 4px 8px; border-radius: 6px; color: white; font-weight: 600; font-size: 12px; }
.red { background:#d9534f; }
.yellow { background:#f0ad4e; color: #111; }
.green { background:#5cb85c; }
.grey { background:#777; }
.panel { margin-bottom: 12px; border:1px solid #e1e1e1; border-radius:6px; overflow:hidden; }
.collapsible { background:#f7f7f7; cursor:pointer; padding:10px 14px; width:100%; border:0; text-align:left; font-size:14px;
font-weight:600; }
.content { padding:10px 14px; display:block; }
table { border-collapse: collapse; width:100%; font-size:13px; }
th, td { border:1px solid #eee; padding:8px; text-align:left; }
th { background:#fafafa; font-weight:700; }
.criticalRow { background-color: #ffe6e6; }
.warningRow { background-color: #fff8e6; }
.healthyRow { background-color: #eef9ee; }
.small { font-size:12px; color:#666; }
</style>
"@

$script = @"
<script>
function togglePanel(id) {
    var el = document.getElementById(id);
    if (!el) return;
```

```
    if (el.style.display === 'none') { el.style.display='block'; }
    else { el.style.display='none'; }
}
window.onload = function() {
    // collapse all content initially
    var contents = document.getElementsByClassName('content');
    for (var i=0;i<contents.length;i++) { /* keep visible for usability */ }
}
</script>
"@

# Header & summary block
$summaryHtml = "<h1>$reportTitle</h1>`n<p class='small'>Generated: $(Get-Date -Format 'yyyy-MM-dd HH:mm:ss')</p>`n"
$summaryHtml += "<h2>Automated Analysis Summary</h2>`n"
$summaryHtml +=
"<table><thead><tr><th>Metric</th><th>Value</th><th>Status</th><th>Target</th></tr></thead><tbody>"
foreach ($r in $analysisRows) {
    $status = $r.Status
    $badge = BadgeHtml $status
    $rowClass = if ($status -eq 'Critical') {'criticalRow'} elseif ($status -eq 'Warning') {'warningRow'} elseif ($status -eq 'Healthy')
{'healthyRow'} else { '' }
    $summaryHtml += "<tr
class='$rowClass'><td>$($r.Metric)</td><td>$($r.Value)</td><td>$badge</td><td>$($r.Target)</td></tr>"
}
$summaryHtml += "</tbody></table>"
```



```
# Build collapsible sections for each report
$sectionsHtml = "<h2>Detailed Sections</h2>`n"
$idx = 0
foreach ($key in $reports.Keys | Where-Object { $_ -ne '__Errors__' }) {
    $idx++
    $id = "sec$idx"
    $sectionsHtml += To-SectionHtml $key $reports[$key] $id
}

# Optional errors
$errorsHtml = ""
if ($reports.ContainsKey('__Errors__')) {
    $errorsHtml = "<h3>Execution Errors</h3><pre>$($reports['__Errors__'])</pre>"
}

# Footer recommendations (simple)
$recommendations = "<h2>Recommendations (auto-generated)</h2><ul>"
if ($pleStatus -eq 'Critical') { $recommendations += "<li><b>Page Life Expectancy is low</b> — consider adding memory or
tuning queries/indexes and investigate buffer pool pressure.</li>" }
if ($ioRStatus -eq 'Critical' -or $ioWStatus -eq 'Critical') { $recommendations += "<li><b>High disk latency</b> — map
problematic files to disks, check disk SKU (Premium/Ultra) and stripe/configuration.</li>" }
if ($metrics.MissingFullBackups.Count -gt 0) { $recommendations += "<li><b>Backups missing or old</b> — validate backup
jobs and perform test restores.</li>" }
if ($metrics.LogFileHighUsage.Count -gt 0) { $recommendations += "<li><b>Log file usage high</b> — consider log backups
frequency or investigate long-running transactions.</li>" }
```

```
if ($metrics.HighWaits.Count -gt 0) { $recommendations += "<li><b>Investigate waits:</b> " + ($metrics.HighWaits -join ", ") + "</li>" }
$recommendations += "<li>Review TempDB configuration: file count, equal sizing, place on low-latency storage.</li>"
$recommendations += "<li>Review MAXDOP and Cost Threshold settings per Microsoft guidance.</li>"
$recommendations += "</ul>"

# Compose final HTML
$finalHtml = "<!doctype html><html><head><meta charset='utf-8'><title>$reportTitle</title>$style</head><body>$summaryHtml $sectionsHtml $errorsHtml $recommendations $script</body></html>"

# Save
Set-Content -Path $htmlPath -Value $finalHtml -Encoding UTF8 -Force

Write-Host "Report generated -> $htmlPath"
if (Test-Path $htmlPath) { Write-Host "Open the file in a browser to view the full report." }
```

3) Quick usage steps

1. Create folder C:\SqlHealthCheck (or your chosen path).
2. Save HealthCheckQueries.sql and Run-HealthCheck-FullReport.ps1 into the folder.
3. Open PowerShell as Administrator on the Azure VM (or admin host with network access to the SQL VM).
4. Install module if needed:
5. Install-Module SqlServer -Scope CurrentUser -Force
6. Run the script (example using Windows auth):
7. cd C:\SqlHealthCheck
8. .\Run-HealthCheck-FullReport.ps1 -SqlInstance "localhost" -OutputFolder "C:\SqlHealthCheck\Output" -UseWindowsAuth

Or for a named instance:

.\Run-HealthCheck-FullReport.ps1 -SqlInstance "myvm\SQL2019" -OutputFolder "D:\Reports" -UseWindowsAuth

9. Open SQL_Azure_HealthCheck_Report.html from the output folder in your browser.

4) Notes & recommended next steps (optional extras I can add if you want)

- Add **email delivery** (Send-MailMessage or Graph API) to send the single HTML file automatically.
- Add **Az PowerShell** collection of VM-level metrics (disk bandwidth, CPU credits, NIC) and embed those charts (requires Az module and Azure permissions).
- Add **threshold customization** as a JSON config consumed by the script.
- Add **scheduling** instructions for Task Scheduler or Azure Automation Runbook.
- Add **DBCC CHECKDB scheduling** (staggered per DB) and capture results to the same HTML.